

11. Write a program for the task of Credit Score Classification.

CODING:

```
import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = {

    'Income': [25000, 30000, 35000, 40000, 45000, 50000, 55000, 60000,

               65000, 70000, 75000, 80000, 85000, 90000, 95000],

    'Age': [22, 25, 28, 30, 32, 35, 38, 40, 42, 45, 48, 50, 52, 55, 58],

    'LoanAmount': [20000, 25000, 30000, 28000, 35000, 30000, 25000, 35000,

                   30000, 25000, 20000, 22000, 18000, 15000, 12000],

    'CreditHistory': [1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1],

    'CreditScore': [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1]

}

df = pd.DataFrame(data)

print("Credit Score Dataset:")

print(df)

X = df[['Income', 'Age', 'LoanAmount', 'CreditHistory']]

y = df['CreditScore']

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.3,

    random_state=42,

    stratify=y

)
```

```

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("\nCredit Score Classification Results")
print("-----")
print("Actual Values :", y_test.values)
print("Predicted Values:", y_pred)
print("Accuracy    :", accuracy)
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=['Poor Credit', 'Good Credit']
))
new_customer = pd.DataFrame({
    'Income': [60000],
    'Age': [35],
    'LoanAmount': [20000],
    'CreditHistory': [1]
})
new_customer_scaled = scaler.transform(new_customer)
prediction = model.predict(new_customer_scaled)

```

```

if prediction[0] == 1:

    print("\nNew Customer Credit Score: GOOD")

else:

    print("\nNew Customer Credit Score: POOR")

```

OUTPUT:

```

Credit Score Dataset:
   Income  Age  LoanAmount  CreditHistory  CreditScore
0   25000   22     20000         1           0
1   30000   25     25000         1           0
2   35000   28     30000         0           0
3   40000   30     28000         1           0
4   45000   32     35000         0           0
5   50000   35     30000         1           1
6   55000   38     25000         1           1
7   60000   40     35000         1           1
8   65000   42     30000         1           1
9   70000   45     25000         1           1
10  75000   48     20000         0           0
11  80000   50     22000         1           1
12  85000   52     18000         1           1
13  90000   55     15000         1           1
14  95000   58     12000         1           1

```

Credit Score Classification Results

```

-----
Actual Values   : [1 1 0 1 0]
Predicted Values: [1 1 0 1 1]
Accuracy        : 0.8

```

Confusion Matrix:

```

[[1 1]
 [0 3]]

```

Classification Report:

	precision	recall	f1-score	support
Poor Credit	1.00	0.50	0.67	2
Good Credit	0.75	1.00	0.86	3
accuracy			0.80	5
macro avg	0.88	0.75	0.76	5
weighted avg	0.85	0.80	0.78	5

New Customer Credit Score: GOOD